COURSE COMPLETED · 2026

LLM Engineering: Master AI & LLMs

An eight-week engineering programme that runs from the first API call all the way to a deployed, autonomous multi-agent product. This is my week-by-week summary of what was built and what it taught me.

Weeks 1–2

Foundations & frontier models

APIs, tokens, streaming, tools, multi-modality and Gradio UIs.

Weeks 3–4

Open source & model selection

Hugging Face, quantization, benchmarks and code generation.

Weeks 5–6

RAG & frontier fine-tuning

Embeddings, Chroma, baselines and a five-step commercial method.

Weeks 7–8

Open-source tuning & agents

QLoRA on a single GPU, then a seven-agent autonomous system.

8	5	7	1
weeks completed	model providers	collaborating agents	deployed product

Talha Rehman · Software Engineer

Instructor: Ed Donner · Udemy

WEEK 01

01 Foundations & the Chat Completions API

How an LLM conversation actually works under the hood — roles, tokens, context windows and streaming.

THE REQUEST / RESPONSE LOOP



The model is stateless — every call must carry the full conversation history

Messages, not prompts

A conversation is an array of system / user / assistant turns sent on every call.

e.g. system sets the behaviour, user carries the question

Stateless by design

The model remembers nothing between calls — history is re-sent each time.

e.g. no history sent = no memory of the last answer

Tokens & context windows

Text is split into tokens; the context window caps how much fits in one call.

e.g. cost and limits are measured in tokens, not words

Streaming responses

Tokens arrive one at a time so the user sees an answer forming immediately.

e.g. the typing effect in every chat product

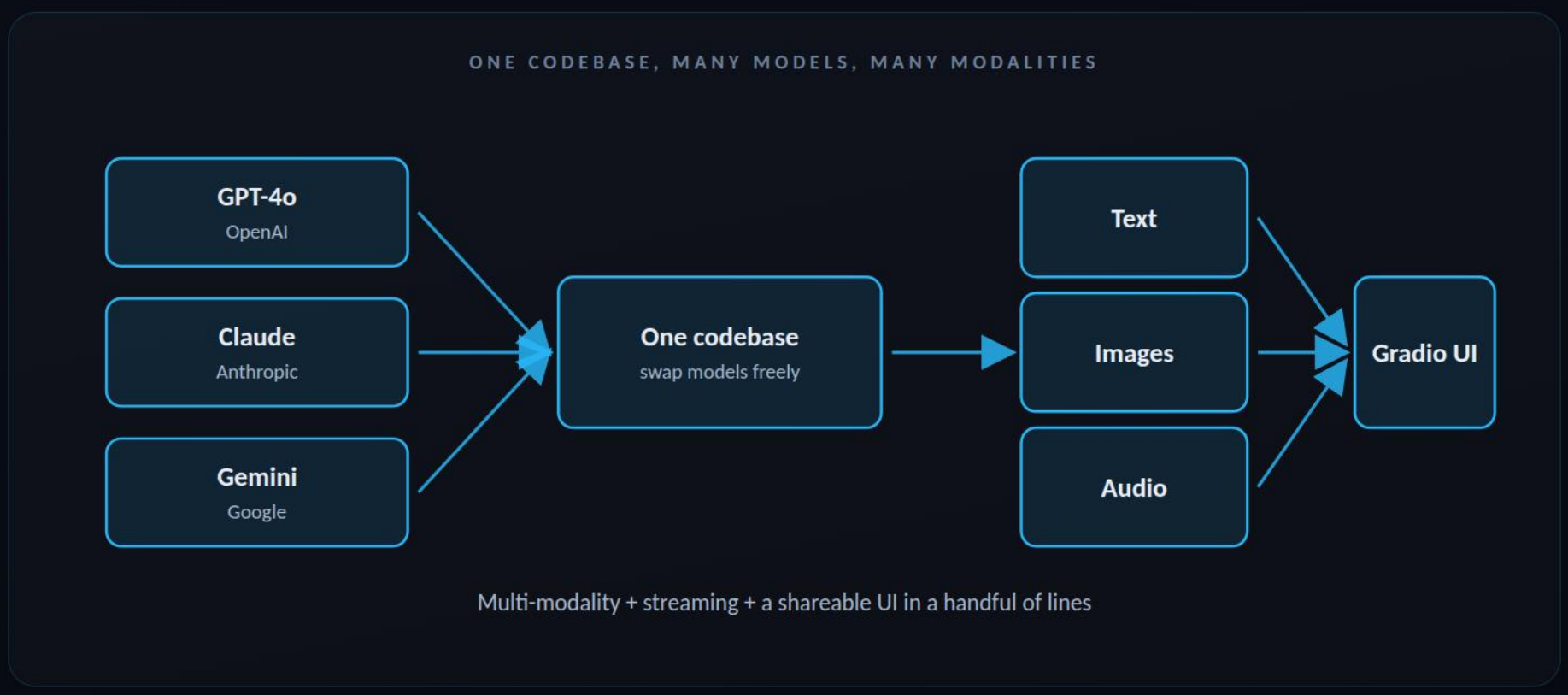
TOOLS & TECHNIQUES

OpenAI API · Anthropic API · Jupyter · Python · dotenv

WEEK 02

02 Frontier Models, UIs & Multi-modality

Calling every major frontier model from one codebase, adding images and audio, and wrapping it in a real interface.



Model-agnostic code

The same request shape works across providers, so models can be swapped freely.

e.g. change one line to move from GPT-4o to Claude

Multi-modality

Models that read images and speak audio, not just text in and text out.

e.g. upload a photo and ask what is wrong with it

Gradio interfaces

A shareable web UI around the model in a few lines of Python.

e.g. gr.Interface(fn=chat).launch()

Tools & function calling

The model decides when to call your Python function and uses the result.

e.g. it calls get_price() instead of guessing

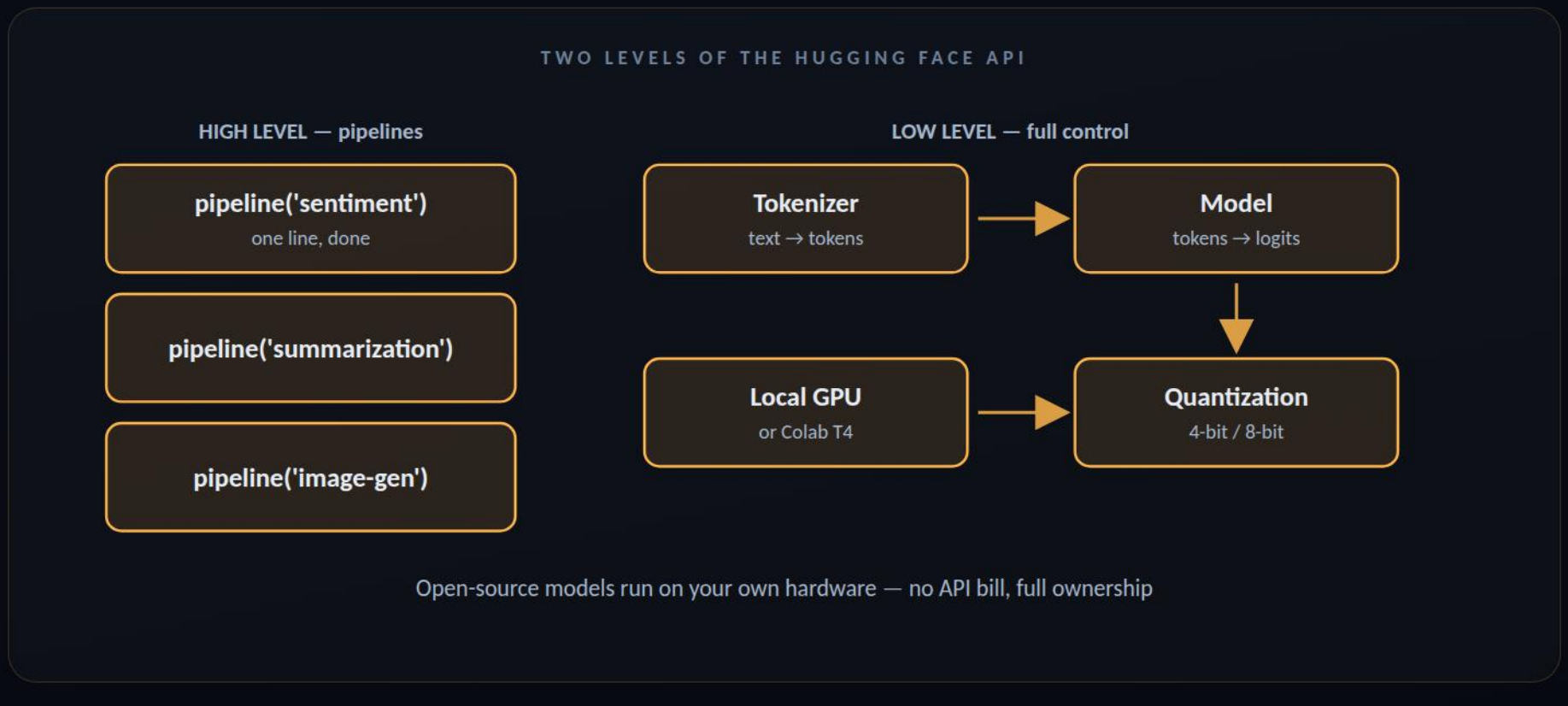
TOOLS & TECHNIQUES

GPT-4o · Claude · Gemini · Gradio · Function calling

WEEK 03

03 Open Source with Hugging Face

Moving off paid APIs — running open models locally and on Colab GPUs, from one-line pipelines down to raw tokenizers.



Pipelines

The high-level API: sentiment, summarisation, translation, image and audio generation.

e.g. pipeline('summarization')(text)

Tokenizers & models

The low-level API — chat templates, generation loops and full control over decoding.

e.g. apply_chat_template() per model family

Quantization

Compressing weights to 4-bit or 8-bit so large models fit on modest GPUs.

e.g. an 8B model on a free Colab T4

The Hub

Models, datasets and Spaces — open weights you own rather than rent.

e.g. no per-token cost once it runs locally

TOOLS & TECHNIQUES

Hugging Face · Transformers · Colab · bitsandbytes · Llama

WEEK 04

04 Selecting LLMs & Code Generation

Choosing the right model with evidence instead of hype, then using LLMs to translate and optimise real code.

SELECTION CRITERIA AND A REAL BENCHMARK



Model choice is an engineering decision — measure, don't guess

Benchmarks and their limits

Leaderboards and arenas are a starting point, not a verdict — they overfit and go stale.

e.g. test on your own task before committing

The real selection criteria

Cost, context window, latency, licence and privacy usually decide before accuracy does.

e.g. a cheaper model that fits the budget often wins

Code generation in practice

Rewriting Python into optimised C++ and measuring the actual speed-up.

e.g. orders of magnitude on the same workload

Frontier vs open-source

Comparing both on identical tasks, with execution as the honest scoreboard.

e.g. run the generated code, don't eyeball it

TOOLS & TECHNIQUES

Benchmarks · Open-source LLMs · C++ · Gradio · Evaluation

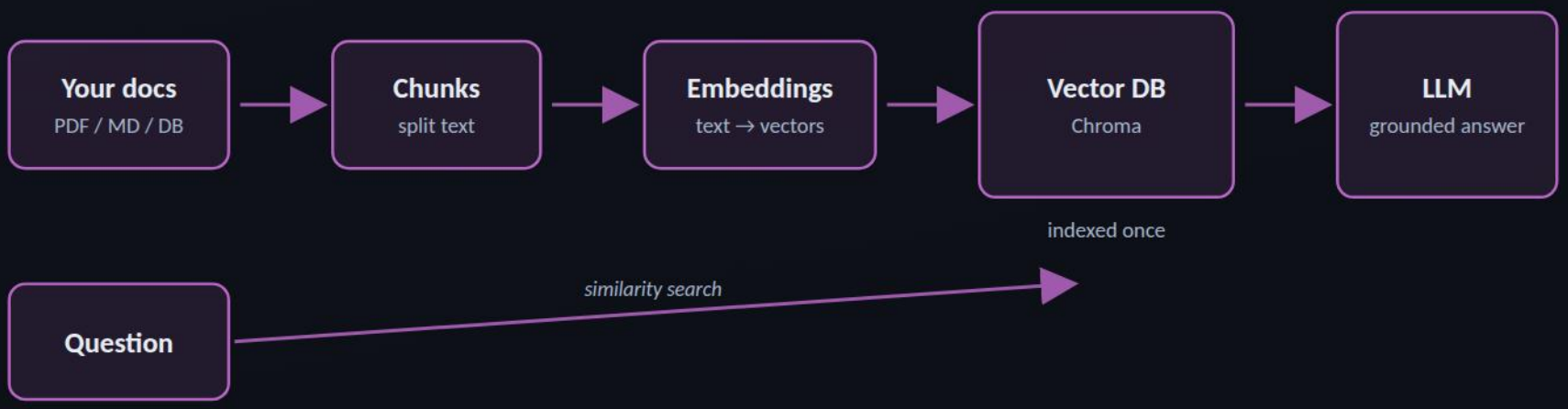
WEEK 05

05 RAG & Question Answering — Creating an Expert

Grounding a model in your own documents with embeddings and a vector database, so answers come from your data.

THE RETRIEVAL-AUGMENTED GENERATION PIPELINE

RAG = retrieve the right context first, then let the model answer from it



Embeddings

Text becomes a vector; similar meanings land close together in that space.

e.g. 'wireless mic' finds 'bluetooth microphone'

Vector databases

Chroma stores the vectors and returns the nearest matches in milliseconds.

e.g. indexed once, queried endlessly

Retrieve, then generate

The retrieved chunks are injected into the prompt so the answer is grounded.

e.g. fewer hallucinations, citable sources

Visualising the vector space

t-SNE projects high-dimensional embeddings to 2D to check they cluster sensibly.

e.g. electronics cluster apart from clothing

TOOLS & TECHNIQUES

Chroma · Embeddings · LangChain · t-SNE · Visualisation

WEEK 06

06 Fine-tuning a Frontier Model

A five-step commercial method applied end to end: curate the data, build honest baselines, then fine-tune and evaluate.

THE 5-STEP METHOD AND THE BASELINE LADDER



The Price Is Right — predict a product's price from its description



Data curation at scale

Cleaning, filtering and balancing hundreds of thousands of real product records.

e.g. the least glamorous work, the biggest lever

Baselines before models

A random guess, then linear regression, then bag-of-words, then ensembles.

e.g. if you can't beat random, stop and look again

Traditional ML still matters

Random Forest and XGBoost on engineered features remain strong, cheap baselines.

e.g. sometimes they beat the LLM outright

When fine-tuning is worth it

It fixes style, format and reliability — it rarely adds world knowledge.

e.g. a clear prompt often beats a fine-tune

TOOLS & TECHNIQUES

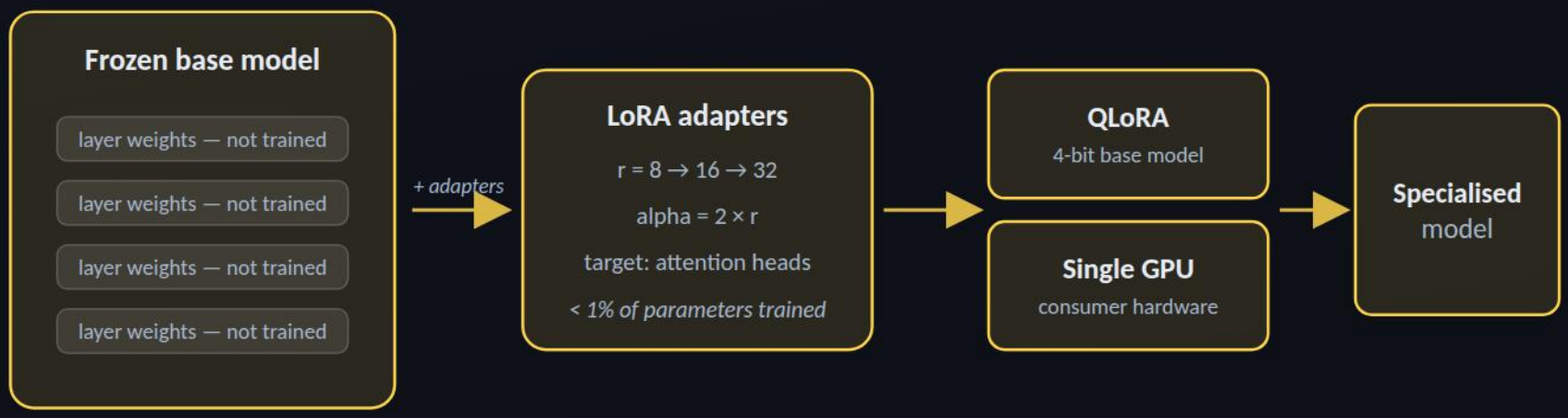
Scikit-learn · XGBoost · Pandas · Structured outputs · Evaluation

WEEK 07

07 Fine-tuning an Open-Source Model

Training a specialised model with QLoRA on a single GPU, and beating a far larger general-purpose model on one narrow task.

HOW LORA AND QLORA ACTUALLY WORK



A small open-source model, fine-tuned for one job, can beat a much larger general model

LoRA

Freeze the base model and train small adapter matrices instead of billions of weights.

e.g. sticky notes rather than rewriting the book

QLoRA

Quantise the base model to 4-bit first, so a large model fits on one consumer GPU.

e.g. the same notes, a much smaller shelf

The three hyperparameters

Rank r for capacity, alpha for influence, target modules for placement.

e.g. start at r=8, alpha=16, attention heads

Base vs instruct models

For a narrow numeric task, a base model avoids inherited chat habits.

e.g. you want a number, not a paragraph

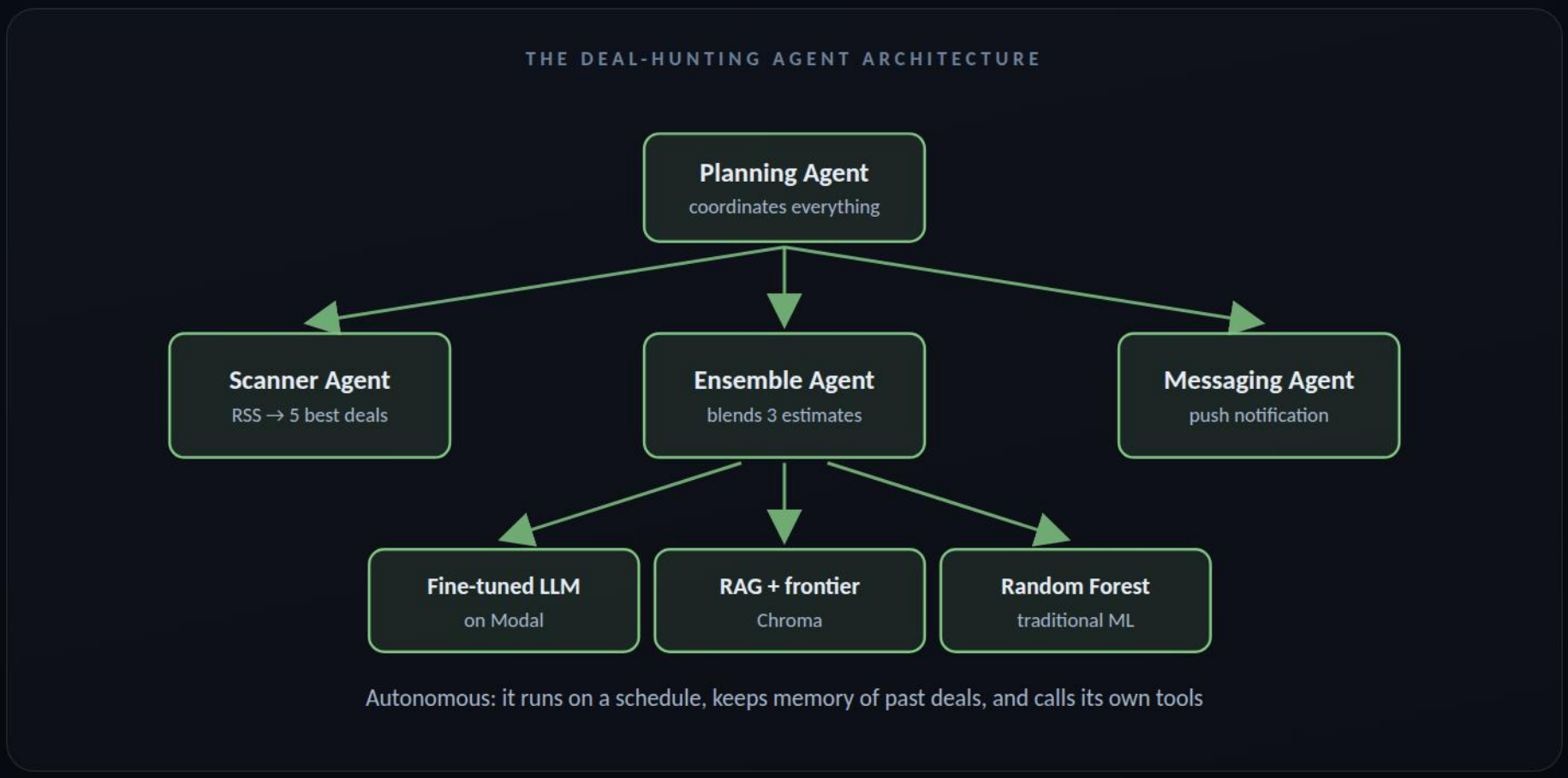
TOOLS & TECHNIQUES

QLoRA · PEFT · Weights & Biases · Llama · Colab GPU

WEEK 08

08 Agentic AI — The Finale

Seven collaborating agents, a fine-tuned model deployed to the cloud, and a system that runs on its own and messages you.



Serverless deployment

The fine-tuned model is deployed to Modal and called remotely like any function.

e.g. pricer.remote(description)

An ensemble of estimators

A fine-tuned LLM, a RAG frontier model and a regression model vote on a price.

e.g. three opinions beat one

Autonomous planning

A planning agent chooses which tool to call, in what order, without being told.

e.g. scan, then value, then notify

Memory & notifications

It remembers what it has already seen and pushes only genuinely new finds.

e.g. runs on a schedule, alerts your phone

TOOLS & TECHNIQUES

Modal · Structured outputs · Chroma · Agents · Gradio · Pushover

THE SHIFT

What eight weeks actually changed

The technical content matters, but the bigger change is in how I approach an AI problem in the first place.

FROM		TO
Calling an API and hoping for the best	→	**Designing prompts, tools and structured outputs**
Trusting a model's first answer	→	**Baselines, metrics and honest evaluation**
One general-purpose model	→	**The right model, chosen on evidence**
Notebooks that run once	→	**Models deployed and callable in production**
A single request / response	→	**Autonomous agents that plan, act and remember**

Next

Applying the same stack with open, free-tier tooling — Groq for inference, sentence-transformers for embeddings, Chroma for retrieval — and rebuilding the agent framework end to end in LangChain and LangGraph.